
PROJET QCM

Par

Amine AMMARI, Zayd BENBOUZID et Alexandre TRAN





Sommaire :

Rapport de Projet Informatique : Gestionnaire de QCM	3
1.Description du sujet et objectifs	4
2.Organisation de l'équipe et flux de travail	5
Répartition des rôles et modularité.....	6
Gestion du code et communication.....	6
3.Problèmes rencontrés et solutions apportées	6
Problème 1 : Les fautes de frappes dans le choix du menu	7
(Saisie de lettres)	7
Problème 2 : Oublier de cocher une bonne réponse à une question	7
Problème 3 : Incohérences logiques lors du paramétrage enseignant	8
4.Résultats obtenus et validation	8
Conclusion.....	9



Rapport de Projet Informatique : Gestionnaire de QCM

Filière : préING1 — CY Tech

Année universitaire : 2025-2026

Application : "Questions pour un Tekien"

Membres de l'équipe :

- Amine
- Zayd
- Alexandre



1. Description du sujet et objectifs

À la suite d'un arrêt technique imprévu de la plateforme Moodle, les enseignants de CY Tech ont exprimé le besoin crucial d'un outil de substitution interactif et automatisé pour concevoir, administrer et évaluer des Questionnaires à Choix Multiples (QCM) à l'approche des partiels.

Notre objectif consistait à développer une application robuste en langage C articulée autour de deux modes distincts :

- **Un Mode Enseignant (sécurisé)** : Permettant le paramétrage complet et la sauvegarde sur fichier de questionnaires personnalisés.
- **Un Mode Étudiant** : Offrant une interface dynamique de sélection, de passage d'examen et de notation automatique sur 20 points.

Le cahier des charges imposait une contrainte stricte de stabilité informatique : l'application ne devait en aucun cas planter ou entrer dans une boucle infinie face aux erreurs de saisie des utilisateurs.

2. Organisation de l'équipe et flux de travail

Afin de mener à bien ce projet en groupe de trois, nous avons mis en place une méthodologie de travail agile et collaborative :

Répartition des rôles et modularité

Pour respecter la consigne de modularité du code, l'architecture logicielle a été découpée en composants indépendants reliés par un fichier d'en-tête commun :

- **Alexandre (main.c et structures.h)** : Responsable de l'arborescence des structures de données (QCM, Question, Choix), du point d'entrée de l'application et de la sécurisation du menu d'aiguillage principal.
- **Amine (enseignant.c et REAdME)** : Chargé du développement de l'interface de création, de la vérification de cohérence des règles du QCM et des algorithmes d'écriture/sauvegarde sur fichiers systèmes.
- **Zayd (etudiant.c , Makefile et qcms)** : En charge de la lecture dynamique des fichiers présents dans le répertoire, de l'affichage de l'examen, de l'implémentation du barème de notation et de l'automatisation du processus de compilation globale.

Gestion du code et communication

- **Dépôt Git Centralisé** : L'intégralité de notre code source a été hébergée sur un dépôt Git distant. Cela nous a permis d'éviter toute perte de données, de suivre l'historique des modifications et de fusionner nos fonctionnalités de manière asynchrone.
- **Environnement et Outils** : Nous avons couplé l'usage de Git avec un serveur Discord dédié aux sessions de programmation hebdomadaires et à la centralisation de nos notes de conception.

3.Problèmes rencontrés et solutions apportées

Au cours des phases d'assemblage et de test de l'application, notre équipe a été confrontée à trois problématiques techniques majeures :

Problème 1 : Les fautes de frappes dans le choix du menu

(Saisie de lettres)

- **Ce qui se passait :** Lorsque le programme affichait le menu principal et demandait un chiffre (1, 2 ou 3), si l'utilisateur tapait une lettre par erreur (comme 'a' ou 'e'), le programme s'emballait. Il affichait le menu en boucle infinie sans jamais s'arrêter, rendant l'application totalement inutilisable.
- **Pourquoi ?** La fonction `scanf` cherche un nombre entier. Si elle trouve une lettre, elle échoue, refuse de la lire et la laisse traîner dans la mémoire de la console (le tampon). Au tour de boucle suivant, `scanf` retrouve la même lettre, échoue à nouveau, et le cycle infernal recommence.
- **La solution :** Nous avons simplement vérifié le résultat de `scanf`. Si la fonction renvoie 0 (ce qui signifie qu'elle n'a pas trouvé de nombre), on utilise une petite boucle pour nettoyer la mémoire de la console et éliminer la lettre fautive avant de réafficher le menu :

```
// Lecture sécurisée du choix utilisateur
if (scanf("%d", &choix) != 1) {
    while (getchar() != '\n'); // Nettoyage du buffer si l'utilisateur tape une lettre
    printf("Saisie invalide. Veuillez entrer un nombre.\n");
    continue;
}
```

Problème 2 : Oublier de cocher une bonne réponse à une question

- **Ce qui se passait :** Lors de la création d'un questionnaire, un enseignant distrait pouvait valider une question en répondant 0 (Non) à l'option "Correct ?" pour absolument tous les choix proposés. Le QCM devenait alors impossible à réussir pour l'étudiant, puisqu'il n'y avait aucune bonne réponse à valider.
- **La solution :** Nous avons simplement ajouté un compteur d'alertes. À chaque choix créé, si l'enseignant coche 1 (Oui), le compteur augmente. À la fin de la question, si ce compteur est toujours égal à zéro, le programme affiche un gros message d'avertissement : " ! ATTENTION : Aucune bonne réponse définie pour cette question". Cela permet au professeur de s'en rendre compte immédiatement.

Problème 3 : Incohérences logiques lors du paramétrage enseignant

- **Symptôme :** Si un enseignant désactivait l'option de réponses multiples mais tentait tout de même de configurer plusieurs réponses correctes au sein d'une même question, la sécurité initiale affichait un message d'erreur mais n'empêchait pas la validation du choix invalide, corrompant les données finales.
- **Solution :** Nous avons remanié le bloc conditionnel au sein de la boucle do...while de vérification. Désormais, en cas d'anomalie détectée (plusieurs choix déclarés vrais alors que `reponsesMultiples == 0`), le mot-clé break est court-circuité. Le programme purge le tampon et force l'utilisateur à ressaisir la conformité de l'option jusqu'à obtention d'une valeur cohérente.

4. Résultats obtenus et validation

L'application finale répond strictement aux exigences formulées dans le cahier des charges :

- **Stabilité (Anti-Crash) :** L'intégralité des flux d'entrée a été blindée. L'introduction accidentelle de caractères alphabétiques à la place de valeurs numériques est interceptée sans provoquer de plantage ni de boucle infinie.
- **Jeux de tests intégrés :** Conformément aux consignes, nous fournissons trois fichiers QCM de démonstration préconfigurés avec des règles variées :
 - DragonBall.qcm : 5 questions, Mode séquentiel, réponses uniques, sans points négatifs.
 - DragonBallZ.qcm : 5 questions, Mode libre, réponses multiples autorisées, avec points négatifs activés.
 - DragonBallSuper.qcm : 7 questions, Mode séquentiel, réponses uniques, avec points négatifs.
- **Compilation industrialisée :** Le Makefile livré automatise la génération séparée des objets (.o) et offre des commandes fiables de réinitialisation (make clean, make fclean, make re) pour faciliter l'évaluation le jour de la soutenance.



Conclusion

Ce projet de première année à CY Tech nous a permis d'assimiler les concepts fondamentaux de la programmation modulaire en C (gestion fine de la mémoire, manipulation des pointeurs, structures imbriquées et flux de fichiers). Au-delà de l'aspect technique, il a constitué une excellente opportunité de parfaire notre méthodologie de travail collaboratif sous Git, garantissant la livraison d'une application stable, structurée et prête pour l'évaluation.